

UNIVERSIDADE FEDERAL DO RIO DE JANEIRO
ESCOLA POLITÉCNICA
DEPARTAMENTO DE ENGENHARIA ELETRÔNICA E DE
COMPUTAÇÃO

PROPOSTA DE PROJETO DE GRADUAÇÃO

Aluno: Matheus Hoffmann Fernandes Santos
hoffmann@poli.ufrj.br

Orientador: A definir, Dr.

Coorientador: Pedro Veras Guimarães, Dr. (LabECO, Departamento de
Engenharia Mecânica, UFSC)

1. TÍTULO

Otimização operacional do modelo de ondas WAVEWATCH III® no ciclo de previsão do ReNOMO (LabECO/UFSC): opções de compilação, configuração, Fortran moderno e reescrita de *kernels* em C++/Kokkos para GPU.

2. ÊNFASE

Computação / Computação científica de alto desempenho.

3. TEMA

Modelos de ondas de terceira geração resolvem a equação de balanço da ação das ondas sobre uma grade espacial e uma grade espectral, integrando em cada ponto e passo de tempo a entrada de vento, as interações não lineares e a dissipação. O WAVEWATCH III® (WW3), desenvolvido pelo NOAA/NCEP, é o modelo de referência dessa classe e o adotado pela maioria dos centros operacionais (The WAVEWATCH III Development Group, 2019). É escrito em Fortran com MPI e OpenMP, e um arquivo de *switches* fixa em tempo de compilação a física, o esquema de propagação e as opções de entrada e saída.

O Laboratório de Engenharia e Ciências Oceânicas da Universidade Federal de Santa Catarina (LabECO/UFSC) executa o WW3 como parte de sua contribuição à Rede Nacional de Observação e Monitoramento Oceânico (ReNOMO), financiada pela chamada CNPq/MCTI/Finep 062/2022, que integra observação, controle de qualidade e produtos operacionais para o Atlântico Sul e a costa brasileira (LabECO/UFSC, 2026). Em um ciclo operacional, o custo de uma previsão é o seu

tempo de execução (*wall-clock*), que determina quantas vezes por dia o modelo pode rodar, em que resolução e com quantos domínios aninhados ou membros de conjunto.

O tema deste projeto é a redução sistemática desse tempo de execução na configuração operada pelo LabECO, tratada como um problema de engenharia com resultados mensuráveis: um *benchmark* reproduzível da execução atual, um perfil que mostre onde o tempo é gasto e uma sequência de otimizações de profundidade crescente, das opções de compilação e de configuração, que não alteram o código, passando pela refatoração dirigida em Fortran moderno, até a reescrita dos *kernels* dominantes em C++/Kokkos para execução em GPU (NVIDIA H100). A ordem é deliberada: as otimizações de configuração e compilação são esgotadas e medidas antes de qualquer reescrita, e cada etapa, inclusive a de GPU, só é aceita se reduzir o tempo de execução sem regressão de resultados dentro de tolerâncias acordadas com os pesquisadores: a cadeia operacional nunca troca correção por velocidade.

4. DELIMITAÇÃO

O trabalho restringe-se ao *driver* de grade única do WW3 (`ww3_shel`) na configuração operada pelo LabECO: sua grade, seu arquivo de *switches*, sua forçante e sua lista de saídas. Essa configuração e a revisão exata do código-fonte são congeladas e documentadas no início do projeto, de modo que todas as medidas se refiram ao mesmo caso.

Quatro etapas estão no escopo, executadas nesta ordem e cada uma validada por comparação com a execução de referência:

1. **Opções de compilação.** Compilador, *flags* de otimização, arquivo de *switches* (em particular memória compartilhada, MPI ou híbrido, e a saída NetCDF-4) e a distribuição de processos MPI e *threads* OpenMP. Não altera o código-fonte; espera-se reprodução bit a bit da referência ou, com reordenação de operações em ponto flutuante, diferenças da ordem do arredondamento.
2. **Configuração da execução.** Decomposição do domínio, passos de tempo permitidos pela condição CFL, frequência e campos de saída, estratégia de *restart* e interpolação da forçante. Altera apenas *namelists*.
3. **Refatoração dirigida em Fortran moderno.** Reescrita das rotinas que o perfil identificar como dominantes, tipicamente a integração dos termos

de fonte (W3SRCE) e a propagação espacial, com construções padrão Fortran 2008/2018 (do `concurrent`, interfaces explícitas, vetores contíguos), sem alterar o algoritmo numérico e dentro de uma tolerância definida por campo de saída.

4. **Reescrita de *kernels* em C++/Kokkos e execução em GPU (NVIDIA H100).** Substituição incremental, de baixo para cima, das rotinas que permanecerem dominantes após a etapa 3 por *kernels* Kokkos (Trott et al., 2022), chamados do Fortran por interfaces `bind(C)` e mantidos no mesmo binário que a rotina original, atrás de uma chave de execução. Começa pelo termo de interação não linear e segue na ordem do perfil. Não é um *port* completo do modelo: o estado permanece no Fortran e o custo de transferência CPU–GPU por passo de tempo é medido e reportado. Um *kernel* em GPU passa à configuração operacional do laboratório se, e somente se, reduzir o tempo de execução do caso operacional e cumprir os mesmos critérios de paridade das etapas anteriores.

A ordem das etapas é parte da delimitação: as etapas 1 e 2 são esgotadas antes de qualquer alteração de código, e a etapa 3 antes da 4, de modo que a reescrita em C++/Kokkos só é empreendida sobre rotinas cujo custo residual já foi demonstrado. O projeto não contribui ao WAVEWATCH IV (WW4) nem depende dele, mas adota a estrutura de testes do WW4 (níveis unitário e de integração em GoogleTest) para os *kernels* reescritos, para que os artefatos sejam aproveitáveis na migração futura. Ficam fora do escopo o *driver* multigrade (`ww3_multi`), acoplamento com outros modelos, assimilação de dados, grades não estruturadas (PDLIB), salvo se a grade operacional as exigir, e alterações nas parametrizações físicas.

5. JUSTIFICATIVA

Uma cadeia operacional de previsão vale pela sua pontualidade. Para o campo de ondas que alimenta os produtos do ReNOMO, um ciclo mais curto significa entrega mais cedo, espaço para maior resolução no mesmo prazo e a possibilidade de rodar vários membros para quantificar incerteza. O WW3 foi projetado para os supercomputadores das duas últimas décadas e carrega, no sistema de compilação, na organização dos dados e na decomposição paralela, escolhas que não são automaticamente as melhores para as máquinas de um laboratório universitário hoje:

nós com muitos núcleos e muita memória, armazenamento NVMe e, cada vez mais, GPUs.

As duas primeiras etapas justificam-se pela documentação do próprio modelo: o arquivo de *switches* e as opções do compilador decidem qual código é compilado e como, e a distribuição paralela decide quanto tempo se gasta movendo espectros entre processos em vez de calculá-los (The WAVEWATCH III Development Group, 2019). São etapas baratas, frequentemente negligenciadas em instalações configuradas uma vez e nunca revisitadas, e produzem o instrumento de que as etapas seguintes dependem: um *benchmark* reproduzível e um perfil. Por isso vêm primeiro: nenhuma reescrita é justificável antes de saber quanto do tempo sobrevive a elas.

A terceira etapa justifica-se pela estrutura do código. Perfis publicados do WW3 mostram que a integração dos termos de fonte domina o tempo de execução, seguida da propagação, e que as rotinas envolvidas usam grandes vetores locais e estado global em nível de módulo, o que dificulta tanto a vetorização quanto qualquer forma de *offload* (Ikuyajolu et al., 2023). Reescrevê-las em Fortran moderno, preservando algoritmo e aritmética, é uma tarefa de engenharia bem compreendida, de valor diretamente mensurável no modelo que o laboratório opera, e prepara a etapa seguinte, pois uma rotina com interface explícita e dados contíguos pode ser substituída por um *kernel*.

A quarta etapa responde à pergunta que o laboratório de fato faz, se um H100 pode acelerar o modelo em operação. O WW3 não tem caminho de execução em GPU na distribuição oficial. O único *port* publicado levou os termos de fonte à GPU com diretivas OpenACC e obteve cerca de 1,3 vez de ganho por nó, limitado pelo tráfego CPU–GPU a cada passo de tempo (Ikuyajolu et al., 2023). Já o WAM6-GPU, que refatorou o modelo inteiro para manter os campos residentes no dispositivo, reportou ganho de uma ordem de grandeza em um nó com várias GPUs (Yuan et al., 2024). A lição é que o *offload* parcial por diretivas, com o estado no *host*, não compensa, e que o ganho vem de *kernels* desenhados para o dispositivo. Este projeto adota o caminho intermediário demonstrado para o FESOM2 por Koldunov et al. (2026), que levaram cerca de 74 mil linhas de Fortran a C e depois a C++/Kokkos em semanas, com um assistente de código baseado em LLM dirigido

por especialistas, tradução literal em dois estágios e validação em escada. Um *kernel* reescrito assim roda em CPU e em GPU a partir do mesmo código e pode ser validado isoladamente contra o Fortran original. O critério para entrar em operação é o mesmo das demais etapas, ganho medido e ausência de regressão, e não uma decisão tomada de antemão.

Estado dos testes do WW3. A validação proposta apoia-se na infraestrutura de testes do próprio modelo, levantada na revisão `develop` 7.14 em setembro de 2026 (NOAA-EMC, 2026). O WW3 tem 62 casos de regressão em `regtests/`, executados por uma matriz de *scripts* que compila combinações de *switches* e compara as saídas de duas execuções arquivo a arquivo com `cmp` e `diff`, classificando cada caso como idêntico ou não idêntico; a matriz inclui variantes específicas de reprodutibilidade bit a bit entre reinícios, número de *threads* e número de processos. Não há comparação com tolerância: qualquer diferença numérica é reportada como falha e cabe ao revisor julgá-la. Testes unitários existem desde 2023, mas cobrem apenas entrada e saída (cinco programas via CTest). A integração contínua pública compila com GNU e Intel e executa um único caso de regressão; a matriz completa roda nas máquinas do NCEP. A atividade recente concentra-se em limpeza de avisos de compilação e em correções da CI, com falhas intermitentes da matriz em aberto. Para este projeto, isso define o método: as etapas 1 e 2 são validadas com a matriz do próprio modelo, cujo critério bit a bit é o adequado; as etapas 3 e 4 exigem um comparador por campo com tolerância e testes unitários por rotina, que o WW3 não tem e que o projeto constrói e entrega.

Estado do WW4. O repositório do WAVEWATCH IV foi criado em novembro de 2025 e, em 15 de setembro de 2026, tem 36 *commits* no ramo `develop`, nenhuma versão publicada e último envio em agosto de 2026 (NOAA-EMC, 2026). A Fase II do projeto (linguagens, governança e arquitetura) foi concluída em março de 2026 (Tolman, 2026); o código atual é um núcleo em C++ com utilitários e um executável autônomo, ainda sem física. A estratégia de testes está definida em quatro níveis, unitário (L1), integração (L2), funcional (L3) e regressão (L4), em GoogleTest com CTest e execução na CI a cada envio; L1 e L2 foram reescritos em julho de 2026 para cobrir todo o código existente, enquanto L3 e L4, que substituiriam a matriz de regressão do WW3, ainda não existem. O sistema de compilação está

sendo simplificado para CMake puro, e a arquitetura CPU–GPU é uma questão em aberto, com Kokkos indicado como camada de abstração de dados e a escolha entre uma ou duas bases de código por decidir. O primeiro lançamento público é esperado para janeiro de 2027, e o plano do NOAA só descontinua o suporte ao WW3 quando o WW4 amadurecer (NOAA/NWS/NCEP, 2025). Duas consequências: o WW3 seguirá como modelo operacional do laboratório por anos, o que justifica otimizá-lo; e os *kernels* reescritos aqui, com Kokkos e testes L1/L2 no padrão do WW4, podem informar a discussão de arquitetura do WW4 em vez de duplicá-la.

6. OBJETIVO

O objetivo geral é reduzir, de forma medida e reproduzível, o tempo de execução do ciclo operacional do WAVEWATCH III mantido pelo LabECO/UFSC no âmbito do ReNOMO, sem alterar seus resultados científicos além das tolerâncias acordadas com o laboratório, esgotando as otimizações de configuração e compilação antes de reescrever código, e decidir com o mesmo critério, ganho medido e ausência de regressão, se *kernels* em C++/Kokkos executados em uma GPU NVIDIA H100 entram na configuração operacional.

Os objetivos específicos são:

1. Congelar e documentar a configuração operacional (revisão do código, *switches*, *namelists*, grade, forçante, saídas, hardware) e construir um *benchmark* reproduzível da execução de referência, com métrica definida (tempo de execução por hora de previsão).
2. Produzir um perfil da execução de referência por rotina e por fase (termos de fonte, propagação, comunicação, entrada e saída), com um e com vários processos MPI.
3. Quantificar o ganho das etapas de compilação, configuração e refatoração em Fortran, cada uma com sua evidência de paridade em relação à referência, e entregar ao laboratório a melhor configuração como um *build* documentado.
4. Construir a infraestrutura de validação que o WW3 não tem: um comparador por campo com tolerâncias versionadas e testes unitários por rotina com entradas capturadas, integrados à matriz de regressão do modelo.
5. Reescrever em C++/Kokkos, na ordem do perfil, os *kernels* que permanecerem dominantes, validá-los contra o Fortran original em CPU, medir seu

ganho em GPU (H100) e decidir, com base em ganho e paridade, se entram na configuração operacional.

6. Publicar ferramentas, resultados e recomendação no repositório aberto, em forma que o laboratório possa reexecutar.

7. METODOLOGIA

O projeto segue o princípio de medir antes de otimizar: nenhuma otimização é tentada antes que a execução de referência seja reproduzível e perfilada, nem aceita sem verificação de paridade contra ela.

Referência e *benchmark*. O caso operacional do laboratório é capturado como um teste autocontido: arquivos de entrada, forçante para um período fixo, arquivo de *switches*, *namelists* e resultados de referência. O mesmo é feito com o teste de regressão oficial mais próximo da configuração operacional. O *script* de *benchmark* registra tempo por fase, processos e *threads*, compilador e opções e a revisão do código, e produz uma tabela anexável aos relatórios.

Perfilamento. A execução de referência é perfilada com *gprof* ou *perf* com um, quatro e dezesseis processos MPI, com o tempo de comunicação registrado separadamente. O resultado é uma tabela de tempo por rotina, agrupada nas fases do laço temporal, que define a ordem da refatoração e a base de todas as medidas posteriores.

Crítérios de paridade. Nas etapas 1 e 2, os resultados são comparados bit a bit com a matriz de regressão do próprio WW3, usada tal como está; quando uma opção do compilador reordena operações em ponto flutuante, a comparação usa tolerâncias por campo (por exemplo, altura significativa, período de pico e direção média), propostas pelo aluno e aprovadas pelo coorientador. Para as etapas 3 e 4, o projeto adiciona o que a matriz não oferece: um comparador por campo que lê as saídas NetCDF de duas execuções e aplica tolerâncias registradas em um arquivo versionado, e testes unitários por rotina, que executam a rotina original e a reescrita sobre entradas capturadas do caso operacional. Uma alteração que falha no critério não é incorporada.

Etapas. As opções de compilação são exploradas primeiro, como uma matriz de compilador, opções e distribuição paralela. Segue a configuração da execução, variando um parâmetro de *namelist* por vez. A refatoração em Fortran ataca as

rotinas do topo do perfil, uma de cada vez, em um *fork* que mantém a rotina original acessível por uma chave de execução no mesmo binário. Só então começa a etapa 4, restrita às rotinas cujo custo residual o perfil atualizado ainda mostrar como dominante.

Interoperabilidade e testes dos *kernels*. Cada *kernel* em C++/Kokkos é exposto ao Fortran por uma interface `ISO_C_BINDING/bind(C)`, e a rotina original permanece no binário atrás da mesma chave de execução, de modo que a matriz de regressão e o comparador rodam os dois caminhos sem recompilar. O leiaute dos *Views* Kokkos reproduz o dos vetores espectrais do WW3, para que a chamada em CPU não copie dados. A tradução segue a receita do FESOM2 (Koldunov et al., 2026): tradução literal para C, verificada contra o Fortran em entradas capturadas; versão Kokkos cujo *backend* serial deve ser idêntico bit a bit ao C; e só então o *backend* CUDA no H100, comparado estatisticamente, com o tráfego CPU–GPU por passo de tempo registrado. Os testes seguem a estrutura do WW4: nível L1, teste unitário do *kernel* sobre um espectro sintético (JONSWAP) com tolerâncias declaradas e justificadas, em GoogleTest; nível L2, reexecução do caso de regressão mais próximo e do caso operacional, comparando WW3 e WW3+Kokkos com o comparador por campo. Um assistente de código baseado em LLM é usado na tradução sob as mesmas regras, com o aluno e o coorientador responsáveis pela validação; *prompts* e evidência de paridade são versionados com o código. A decisão de levar um *kernel* à operação é tomada com o LabECO a partir de duas evidências: a tabela de tempo do caso operacional, com e sem o *kernel*, e o relatório de paridade.

Ferramentas e reprodutibilidade. Todo o trabalho é feito no repositório aberto `ww3-gpu` (Hoffmann, 2026), que fornece a cadeia de ferramentas fixada, os *scripts* de compilação e de teste de regressão, o modelo como submódulo em revisão fixa e integração contínua, de modo que qualquer medida pode ser repetida pelo laboratório em suas próprias máquinas. Os resultados são registrados como tabelas e gráficos gerados por *scripts*, nunca à mão.

Entregas e validação. As entregas são o registro de configuração, as tabelas de *benchmark* e de perfil, o melhor *build* documentado, o comparador por campo e os testes unitários, os *kernels* Kokkos com sua evidência de paridade e tempo, e o relatório final com a recomendação de operação. A validação científica cabe ao

coorientador no LabECO, que define as tolerâncias e revisa a evidência de paridade; a de engenharia, pelos testes automatizados do repositório e por reexecuções independentes no laboratório.

Riscos. O principal é a divergência silenciosa de resultados após uma reescrita, mitigada pelos critérios de paridade e pelos testes por rotina. O segundo é a etapa 4 consumir o tempo das anteriores, mitigado pela ordem fixa das etapas e pela regra de só reescrever rotina cujo custo residual foi medido. O terceiro é o ganho em GPU ficar limitado pelo tráfego de dados enquanto o estado permanece no Fortran; nesse caso o resultado é a medida desse limite e a recomendação de não operar em GPU. O quarto é a grade operacional ser não estruturada; nesse caso as etapas 3 e 4 restringem-se aos termos de fonte.

8. CRONOGRAMA

O projeto ocupa dois períodos letivos; as datas serão ajustadas com os orientadores.

Etapa	Prazo
Revisão bibliográfica; registro da configuração operacional	10/2026
<i>Benchmark</i> reprodutível e perfil da execução de referência	11/2026
Etapa 1: opções de compilação (compilador, <i>flags</i> , <i>switches</i> , paralelismo)	12/2026
Etapa 2: configuração da execução; comparador por campo e testes por rotina	01/2027
Etapa 3: refatoração dirigida em Fortran moderno das rotinas dominantes	02/2027
Etapa 4: <i>kernels</i> em C++/Kokkos; paridade em CPU e medição no H100	03/2027
Decisão de operação com o LabECO; relatório final e revisão	04/2027
Defesa (a partir de)	05/2027

Referências Bibliográficas

HOFFMANN, Matheus. **ww3-gpu: an open lab for WAVEWATCH III operational optimisation and GPU kernels**. <https://github.com/hoffmann/ww3-gpu>, 2026.

IKUYAJOLU, Olawale James *et al.* **Porting the WAVEWATCH III (v6.07) wave action source terms to GPU**. *Geoscientific Model Development*, v. 16, p. 1445–1458, 2023.

KOLDUNOV, Nikolay V. *et al.* **An Ocean Model Ported by a Large Language Model: Experience and Lessons from FESOM2 (Fortran to C to C++/Kokkos)**. arXiv:2606.11356, 2026.

LABECO/UFSC. **ReNOMO – Rede Nacional de Observação e Monitoramento Oceânico**. <https://labeco.ufsc.br/renomo/>, 2026.

NOAA-EMC. **WAVEWATCH III source code repository**. <https://github.com/NOAA-EMC/WW3>, 2026a.

NOAA-EMC. **WAVEWATCH IV (WW4) repository**. <https://github.com/NOAA-EMC/WW4>, 2026b.

NOAA/NWS/NCEP. **WAVEWATCH IV: plan for the next-generation wave model**. [S.l.]: NCEP, 2025.

THE WAVEWATCH III DEVELOPMENT GROUP. **User manual and system documentation of WAVEWATCH III version 6.07**. College Park, MD, USA: NOAA/NWS/NCEP/MMAB, 2019.

TOLMAN, Hendrik L. **The WAVEWATCH IV Project: Phase II report**. [S.l.]: NOAA/NWS/NCEP/EMC, 2026.

TROTT, Christian R. *et al.* **Kokkos 3: Programming model extensions for the exascale era**. **IEEE Transactions on Parallel and Distributed Systems**, v. 33, n. 4, p. 805–817, 2022.

YUAN, Ye *et al.* **WAM6-GPU v1.0: GPU-accelerated third-generation wave model**. **Geoscientific Model Development**, v. 17, p. 6123–6136, 2024.

Rio de Janeiro, 13 de setembro de 2026.

Matheus Hoffmann Fernandes Santos - Aluno

A definir, Dr. - Orientador

Pedro Veras Guimarães, Dr. - Coorientador